

Understanding Santorini: Evolving Interpretable Heuristics with Regularised Genetic Algorithms

Blake Robert Bodycote

May 13, 2025

Abstract

Santorini is an abstract board game characterised by perfect information, determinism, and a exponentially large game tree, with up to 128 possible moves on each player’s turn. While prior work has established its PSPACE-hardness and complex AI solvers, there has been little work achieved on explainable and efficient models. This study applies a regularised genetic algorithm to evolve linear combinations of nine strategic features, such as worker height, centrality, mobility, coverage and proximity and coverage. This is implemented with a depth-3 Negamax solver with alpha-beta pruning and move ordering for smarter decision-making. Over successive generations, agents compete in single-elimination tournaments, with selection, uniform crossover, and Gaussian-noise mutation driving the optimisation of feature weights. We analyse the evolution of mean, and standard deviation of the evolving weights across 200 generations to identify which heuristics most significantly enhance performance. Our results reveal consistent favourability of height, centrality, and proximity features, with biases introduced by limited search depth and feature scaling. This paper discusses the implications of these findings for heuristic design and outline avenues for future work, including variable-depth search and feature rescaling and non-linear transformations.

1 Introduction

1.1 Motivation & Background

Abstract board games like Chess, Go, and Santorini achieve a certain beauty by adhering to two core principles: (1) The game is deterministic — every move when played will yield the same result, and (2) complete information shared between both players. These games are the purest form of a “battle-of-wits” — every possible move is predetermined with both players fighting for the same objective; the challenge lies in selecting the right path to victory.

While much has been studied about optimal play in Chess and Go, less attention has been given to understanding what makes a position favorable in games like Santorini.

Geissler’s independent study proves that Santorini is PSPACE-hard [Geissler, 2020], meaning it is as difficult than the most challenging problems in the PSPACE class, highlighting the computational complexity of the game. PSPACE is the class of problems solvable by a Turing machine in polynomial space. Since each player’s turn involves choosing one of two workers to move to up to 8 adjacent spaces and then build on up to 8 adjacent tiles, the branching factor of this exponentially-growing game tree grows more rapidly than other abstract board games. This is without including the 55 asymmetric “God Powers” available to both players to change the style of play [Roxley Games, 2016].

Multiple AIs have also been developed by individuals for Santorini, with the most accessible being Schneider’s Erastus [Schneider, 2022], which uses a Monte-Carlo Tree Search and includes implementation for parallel computing. However, these models are limited in terms of their explainability - there has been little investigation into which heuristic functions significantly improve the performance of a Santorini AI.

The end of Geissler’s independent study [Geissler, 2020] proposes several heuristics that may be valuable for Santorini, such as the height of the workers, their proximity to the centre of the board, and the distance between workers. However, these heuristics are presented hypothetically without empirical validation. Boreham’s report [Boreham, 2020] conducts some evaluation of heuristic functions with varying parameters by running random games between two preset models. However, it does not include any deep analysis of the weighting of these functions.

This paper builds upon these gaps by applying a genetic algorithm to Santorini, which evolves agents that evaluate board states using differing combinations of strategic metrics. Through generations of competition and selection, these agents gradually converge toward stronger evaluations — driven by the principle of "survival of the fittest" [Moscato and Berretta, 2025b]. Each agent will also have a depth-3 Negamax algorithm to guide play. The primary objective is not necessarily to create a perfect player, but to study the evaluation heuristics that lead to stronger, more consistent play over time.

1.2 Rules of Santorini

Santorini is played on a 5x5 grid, where each player begins by placing two workers on unoccupied spaces, as no space can contain more than one worker. On each turn, a player must move one of their workers to an adjacent space (orthogonally or diagonally), and then build on an adjacent space.

Building increases the level of a space, stacking up to a maximum of 3 levels. On a player's turn, if they build on a space that has reached level 3, they may place a dome instead of a regular building block, capping the tower and rendering that space unusable for movement.

When moving, workers may ascend only one level at a time, but they may move across or descend any number of levels, so long as the space is adjacent. The objective of the game is to be the first player to move one of their workers onto the third level of a building. Additionally, if a player is forced into a position where they cannot move any of their workers on their turn, that counts as a win for the opposing player. [Roxley Games, 2016]

Santorini also provides an more complex versions of play with three-player mode with six workers, or by providing players with an asymmetric "God Power". However, this report will only focus on the two-player mode with no God Powers.

2 Methodology & Introduction

2.1 The Negamax Algorithm

For each agent to evaluate the next move, they must first be provided a representation of the current board state. The position is represented by a collection of bitboards, which are a compact data structure that represents an integer as a an array of binary values for fast operations [Korzh, 2020]. Since the Santorini board is a 5x5 grid, each bitboard is a 25-bit integer, where each bit corresponds to a specific tile on the board and indicates the presence or absence of a particular object at that location. The `Position` class has 5 bitboard for the board storing the levels 0-4, and 2 bitboards for the workers positions, with an extra helper array storing the exact positions for workers for constant time access.

The `Solver` class, which represents an individual agent, contains a Negamax algorithm that searches the game tree to determine which move gives the highest evaluation score [Pons, 2016]. To improve efficiency, this algorithm implements alpha-beta pruning [Wikipedia Contributors, 2025] and move ordering heuristics [Pons, 2016]. In particular, move ordering prioritises actions that place the worker on the highest available levels first, since a worker must stand on level 2 to reach level 3, level 1 to reach level 2, and so on, increasing the likelihood of faster access to a winning position.

The evaluation function used by each solver is a linear combination of nine features, each weighted differently. These weights are either randomly initialised between 0–100 at the start of the program (for the first generation), or passed and mutated from the previous generation in the genetic algorithm.

The nine features and their respective calculations are as follows. Note that the values they are multiplied are arbitrarily such that they cannot exceed the range of [-50, 50]:

1. **currHeight**: Sum of the current player's worker heights, multiplied by 10.
2. **oppHeight**: Sum of the opposing player's worker heights, multiplied by -10.
3. **currMobility**: Number of distinct adjacent move options (excluding overlaps) for the current player's workers, multiplied by 3.
4. **oppMobility**: Number of distinct adjacent move options (excluding overlaps) for the opposing player's workers, multiplied by -3.

5. **currCoverage**: Number of unique spaces where the current player can build this turn, multiplied by 2.
6. **oppCoverage**: Number of unique spaces where the opposing player can build this turn, multiplied by -2.
7. **currCentrality**: For each current player worker: +20 if on the centre tile, +10 if on any of the surrounding 8 inner tiles.
8. **oppCentrality**: For each opposing player worker: -20 if on the centre tile, -10 if on any of the surrounding 8 inner tiles.
9. **proximity**: For each opponent worker, calculate the Chebyshev distance to the nearest current player worker (the maximum of row and column differences), then sum the negative of these distances.

2.2 The Evolutionary Algorithm

The **Environment** class begins by generating a population of agents, each initialised with random weights from the range [0, 100) to be assigned to each feature.

The agents are then evaluated in groups of four using a single-elimination tournament format, where each match is a best-of-one game. The winner from each tournament is decided as the most fit agent and is selected for the next generation. After the best agents are calculated, the mean, median and standard deviation of each of the weights are recorded to analyse how feature importance changes across generations.

A **Game** class is created to coordinate the interaction between the agents playing. It alternates the control of the game state between the two agents, each of which selects its next move using its internal solver, until one of them detects an inevitable win or loss.

Next, the best agents are kept and the rest of the agent pool is refilled via uniform crossover of two randomly selected agents. Then, each agent has a 10% chance of undergoing mutation, where each weight in a selected agent will have a 50% chance to be altered by a random value from a normal distribution of mean -5 and standard deviation 10. Post mutation, the weights are then clamped back into the range of [0, 100). [Moscato and Berretta, 2025a]

The use of a negative mean introduces a regularisation effect, supporting weight decay over time. However, agents whose reduced weights lead to poor performance are naturally eliminated through the fitness-based selection process. This evolutionary cycle is repeated for a fixed number of generations, with the goal of progressively evolving more effective agents.

2.3 Method

This report is interested in analysing two concepts. (1) How the coefficients of the agents are evolving overtime, and (2) what are the most important coefficients to the agents for the last generation. This experiment will select a population size of 100 to be evolved over 200 generations. To study the first concept, one test tracks the mean and standard deviation of each coefficient among the top 25 agents at every 10th generation, including their initial values before the first survival-of-the-fittest selection. Additionally, all five tests will record the coefficients of the top 25 agents in the final generation to address the second concept.

3 Results

3.1 Coefficient Evolution Over Time

In Test 1, the coefficient mean values for all features begin at approximately 50 before diverging over time. Notably, the weights associated with both mobility and coverage (across both current and opponent contexts) gradually decline to zero, while the remaining coefficients maintain relatively stable trajectories with a slight downward trend. The corresponding standard deviation plot shows that all coefficients start varying from 25–30, decreasing gradually to below 15 by the last generation. However, this decrease in the standard deviation paths are jagged, showing the variance fluctuate across epochs.

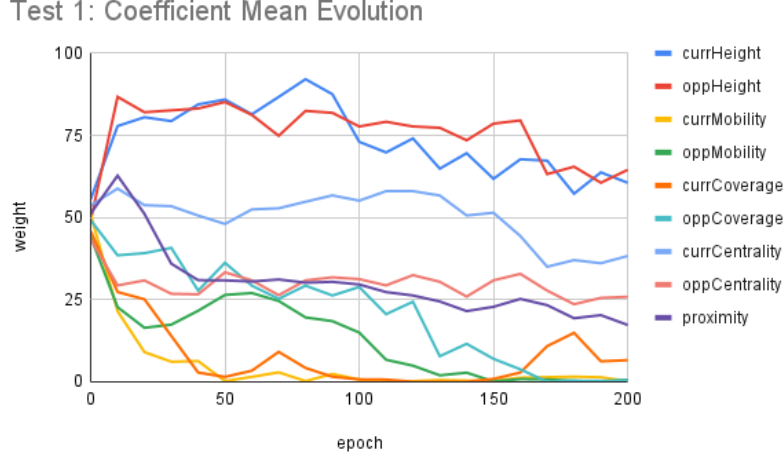


Figure 1: Evolution of the mean values for each feature’s coefficient over 200 training epochs during Test 1.

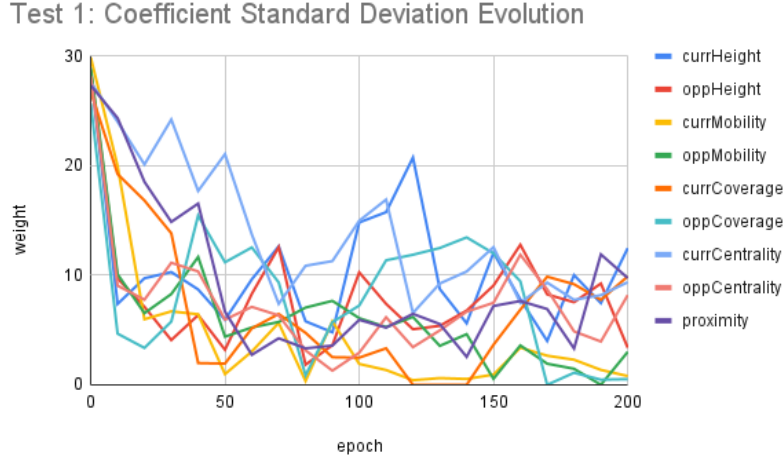


Figure 2: Evolution of the standard deviation of each features’ coefficient over 200 training epochs during Test 1.

3.2 Final Generation Coefficients

For the results, only plots 2, 4, and 5 are shown since they display the most varying results for discussion. All plots for the final generation’s coefficients (including test 1 and 3) are available in the appendix.

From these plots, the weights for **currMobility**, **oppMobility**, and **currCoverage** remain close to zero in all tests, suggesting minimal contribution to the AIs decision-making. In contrast, **currHeight**, **oppHeight**, **currCentrality**, and **oppCentrality** consistently show weights above zero, indicating greater importance, with height-related parameters generally receiving higher coefficients than centrality-related ones in all plots. Parameters prefixed with **opp** tend to have higher weights than their **curr** counterparts, suggesting a bias towards evaluating the opponent’s pieces. **proximity** varies substantially in its weight across all 5 plots. While **oppCoverage** is close to zero in most cases, it shows a distinct spike in Test 2, and has the most variability among the otherwise negligible-weight parameters.

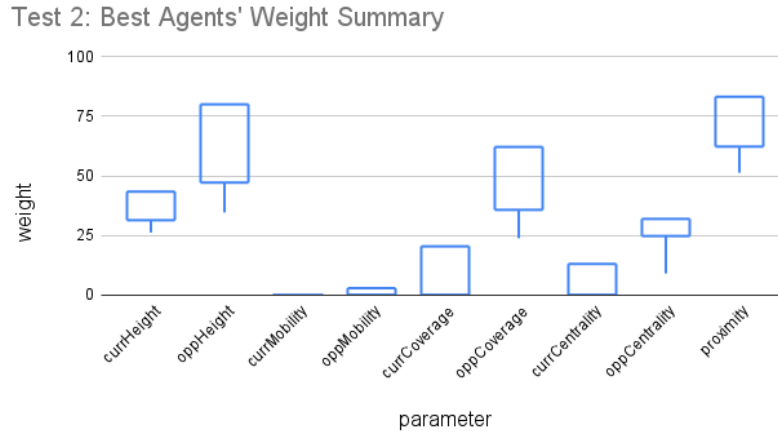


Figure 3: Weight spread summary of each parameter for the final generation's best agents (Test 2).

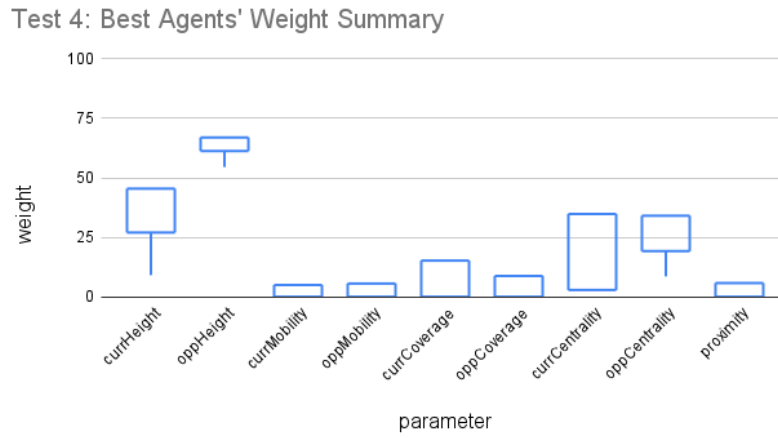


Figure 4: Weight spread summary of each parameter for the final generation's best agents (Test 4).

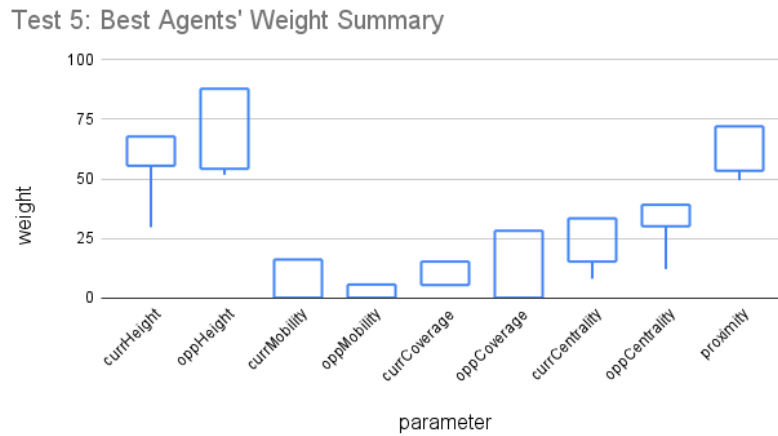


Figure 5: Weight spread summary of each parameter for the final generation's best agents (Test 5).

3.3 Discussion

The regularised genetic algorithm employed for this project shows some consistent results of certain favourability of the parameters. However, there could be some potential flaws in the implementation that some of the previously mentioned trends result from.

Since each agent uses a depth-3 Negamax algorithm to guide its decision-making, it only evaluates the game up to the current agent’s next move, without considering the opponent’s follow-up response in full. This limited foresight may contribute to a bias in the learned weights, where opponent-related coefficients are consistently weighted more heavily. One possible explanation is that the model overestimates the importance of immediate threats or opportunities visible within its shallow search, while underestimating long-term positioning. To mitigate this, a Monte-Carlo Tree Search approach could be used to enable variable-depth exploration, allowing the agent to sample a broader range of plausible future outcomes. This would likely yield a more representative and balanced evaluation of the game state.

Another notable concern in the current approach is how agent coefficients are initialised and interpreted. Since all parameters, features that vary less — even if strategically important — are easily overshadowed. For example, `currCoverage` might offer a meaningful decision-making signal, but if its numerical difference between board states is smaller than that of `currHeight`, it’s less likely to influence early generations. This issue arises not from the arbitrary scaling of each feature to ensure their contributions stay within the range $[-50, 50]$. This could be fixed through proper feature scaling. For example, by increasing the scaling factor of `currCoverage` and offsetting the score with a negative baseline to a value it is highly unlikely to reach, the relative differences between states is more pronounced while still respecting the overall constraint on the output’s magnitude.

There also could be biases caused by treating some of the scaling factors as linear for the parameters. For example, the linearity assumption of `currHeight` means the difference between levels 0 and 1 is equivalent to that between 1 and 2 — a simplification that may not reflect Santorini’s strategic depth. However, finding the correct reward differences for these heights would require complicating the model with more parameters, potentially introducing worse results or overfitting. Balancing simplicity with expressiveness is key, and exploring specific complex scaling options could offer a middle ground of introducing non-linearity without significantly increasing model complexity.

There are also some improvements that could be made to boost the results of this experiment. The primary issue with this testing scheme is the lack of a comparison to evaluate how effective these models are relative to each other. Since the randomised agents are introduced only at the start of the genetic algorithm, the genetic diversity decreases in the later stages, adding a human testing element could help detect flaws in the agents’ reasoning, potentially revealing new parameters to improve the model’s performance. Additionally, the pool of the best agents from each completed test were not pitted against each other, leaving no clear indication of how much better one model is compared to another.

There is also room for experimentation to introduce more complexity into the model. One idea that formed post-experimentation was to include a feature that considers the height of the surrounding terrain directly adjacent to a worker. This would guide the worker towards the highest point, potentially reducing the time spent building upwards. Additionally, incorporating a time coefficient would allow us to observe how strategies evolve over time, providing insights into whether the coefficients change from the early to the late game.

4 Conclusion

This paper demonstrates that a regularised genetic algorithm can help identify the importance of certain evaluation heuristics for Santorini agents, resulting in an interpretable model. Through competitive self-play and statistical tracking across generations, the features related to worker elevation, centrality, and proximity play a critical role in shallow-depth play. However, the depth-3 Negamax restricts foresight, over-emphasising short-term gains and under-representing long-term positional decisions. Additionally, the assumption of linearity in features and lack of genetic diversity and feature scaling introduce biases that can may have suppressed more subtle but strategically meaningful metrics. To address these limitations, future work should incorporate variable-depth exploration with Monte-Carlo Tree Search and employ appropriate feature scaling or non-linear transformations to find

more valuable heuristics. Finally, human evaluation and agent benchmarking would provide more rigorous validation of the quality of the agents’ strategies. By refining both search and feature representation, subsequent studies can advance towards smarter and more interpretable Santorini AIs, as well as inform heuristic design in other complex board games like Chess and Go.

5 Bibliography

References

- [Boreham, 2020] Boreham, J. E. (2020). Developing an artificial intelligence capable of playing the board game santorini. https://info.bb-ai.net/student_projects/project_reports/Games/Jack-Boreham-Santorini-project-report.pdf. Accessed: 2025-05-04.
- [Geissler, 2020] Geissler, C. C. (2020). A mathematical analysis of the game of santorini. <https://openworks.wooster.edu/cgi/viewcontent.cgi?article=11834&context=independentstudy>. Accessed: 2025-05-04.
- [Korzh, 2020] Korzh, M. (2020). Bitboard chess engine in c: creating comfortable conditions for development. <https://www.youtube.com/watch?v=o-ySJ2EBarY>. Accessed: 2025-05-04.
- [Moscato and Berretta, 2025a] Moscato, P. and Berretta, R. (2025a). Evolutionary algorithms: Genetic algorithms part i. Accessed via the 2025 COMP3330 Canvas Page. Lecture notes, COMP3330 – Machine Intelligence, accessed 2025-05-04.
- [Moscato and Berretta, 2025b] Moscato, P. and Berretta, R. (2025b). Evolutionary algorithms: Introduction to evolutionary computation. Accessed via the 2025 COMP3330 Canvas Page. Lecture notes, COMP3330 – Machine Intelligence, accessed 2025-05-04.
- [Pons, 2016] Pons, P. (2016). Solving connect 4: how to build a perfect ai. <http://blog.gamesolver.org/>. Accessed 2025-05-04.
- [Roxley Games, 2016] Roxley Games (2016). Santorini rulebook. <https://cdn.1j1ju.com/medias/fc/ec/5d-santorini-rulebook.pdf>. Accessed: 2025-05-04.
- [Schneider, 2022] Schneider, R. (2022). Erastus: Santorini ai. <https://github.com/richardjs/erastus>. Accessed: 2025-05-04.
- [Wikipedia Contributors, 2025] Wikipedia Contributors (2025). Negamax - negamax with alpha beta pruning. https://en.wikipedia.org/wiki/Negamax#Negamax_with_alpha_beta_pruning. Accessed 2025-05-04.

A Plots



Figure 6: Weight spread summary of each parameter for the final generation's best agents (Test 1).

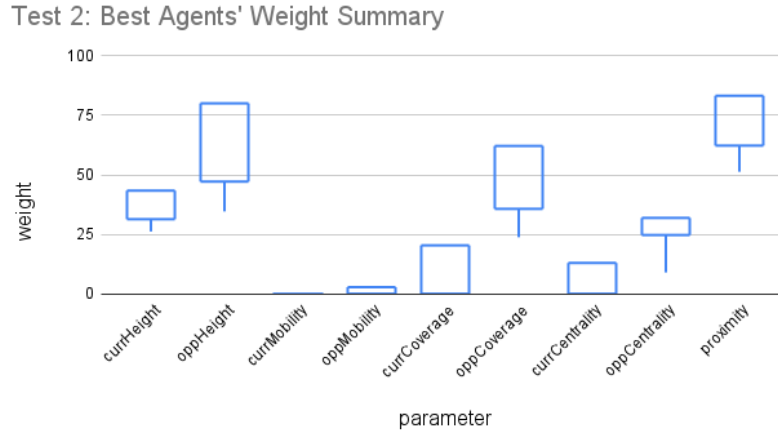


Figure 7: Weight spread summary of each parameter for the final generation's best agents (Test 2).

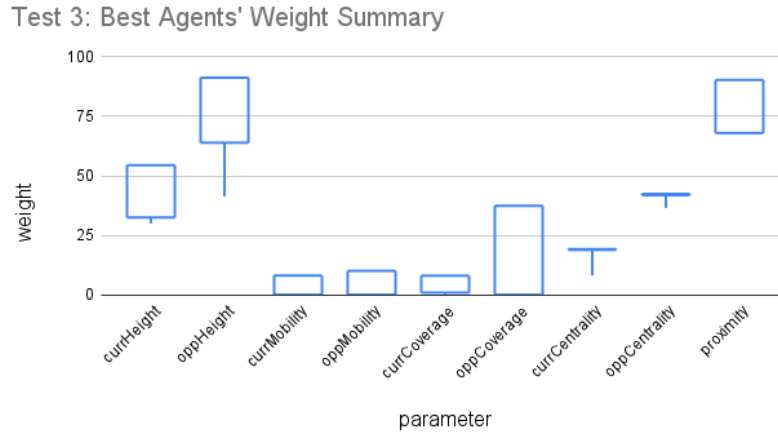


Figure 8: Weight spread summary of each parameter for the final generation's best agents (Test 3).

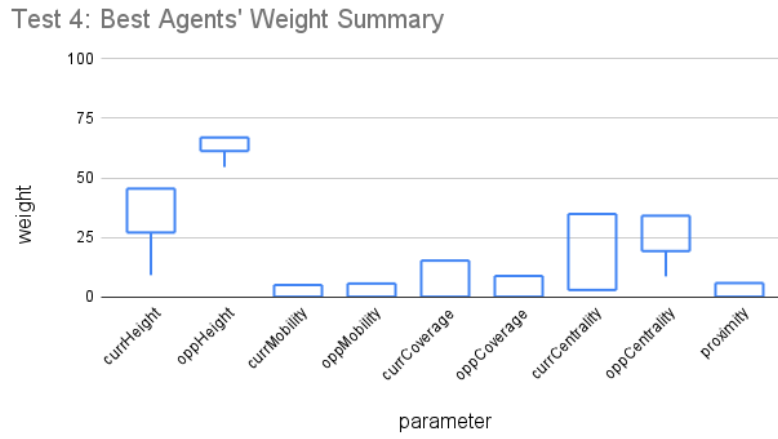


Figure 9: Weight spread summary of each parameter for the final generation's best agents (Test 4).

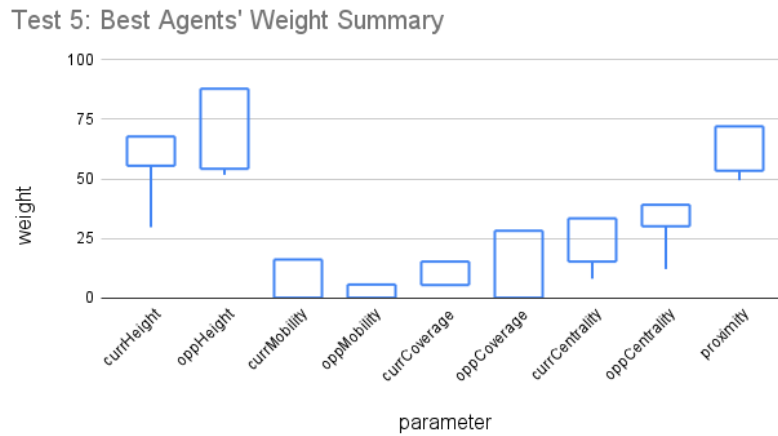


Figure 10: Weight spread summary of each parameter for the final generation's best agents (Test 5).